Operating Systems (UE24CS242B)

Orange Problem 2 Date: 17/04/2026

Name: Shashidharan VS	SRN: PES2UG24CS568	Section: 4J

1A.

```
make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B.

```
All Phase 1 tests passed.
find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

2A.

```
make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Operating Systems (UE24CS242B)

Orange Problem 2 Date: 17/04/2026

Name: Shashidharan VS	SRN: PES2UG24CS568	Section: 4J

2B.

```
All Phase 1 tests passed.
find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
```

3A.

```
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects.c
  untracked:  object.c
```

3B.

```
cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776445529 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776445529 6 file2.txt
```

Operating Systems (UE24CS242B)

Orange Problem 2 Date: 17/04/2026

Name: Shashidharan VS	SRN: PES2UG24CS568	Section: 4J

4A.

```
./pes log
Initialized empty PES repository in .pes/
Committed: f7c40ad79998... Initial commit
Committed: 120a8b67291a... Add world
Committed: 35113b2831f3... Add farewell
commit 35113b2831f34aad1ad36a871a9870fa665112cc15b3936976ee0147195ddc6c
Author: shashi <PES2UG24CS568>
Date:   1776445835

    Add farewell

commit 120a8b67291a8cb05f91a6aeb19558245df5a6bbd59115228ade6128db77c406
Author: shashi <PES2UG24CS568>
Date:   1776445835

    Add world

commit f7c40ad79998971d32af6cd7b2fa8612c4823753ecd6e77716f7e12261adf7d0
Author: shashi <PES2UG24CS568>
Date:   1776445835

    Initial committial commit
```

4B.

```
find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/12/0a8b67291a8cb05f91a6aeb19558245df5a6bbd59115228ade6128db77c406
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/35/113b2831f34aad1ad36a871a9870fa665112cc15b3936976ee0147195ddc6c
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/objects/f7/c40ad79998971d32af6cd7b2fa8612c4823753ecd6e77716f7e12261adf7d0
.pes/refs/heads/main
```

Operating Systems (UE24CS242B)

Orange Problem 2 Date: 17/04/2026

Name: Shashidharan VS	SRN: PES2UG24CS568	Section: 4J

4C.

```
cat .pes/refs/heads/main
35113b2831f34aad1ad36a871a9870fa665112cc15b3936976ee0147195ddc6c
cat .pes/HEAD
ref: refs/heads/main
```

5.1: To implement pes checkout <branch>, several updates must be handled carefully. First, verify that the branch reference exists inside .pes/refs/heads/. Then update .pes/HEAD to point to that branch and retrieve its latest commit hash. The system must load the corresponding commit tree, update the working directory to match it, and sync the index accordingly. The most complex part is safely rewriting the working directory without losing uncommitted changes.

5.2: Detecting a dirty working directory requires a three-step comparison process. Begin by comparing current files with the .pes/index, using metadata and content hashes when necessary. Next, compare the current branch tree with the target branch tree to identify differing paths. If any file is both locally modified and differs between branches, the checkout must be aborted to prevent accidental data loss.

5.3: A detached HEAD state occurs when HEAD points directly to a commit hash instead of a branch reference. In this mode, new commits can still be created, but they are not associated with any branch name. This makes them harder to locate later if the user switches contexts. To recover such commits, a branch can be created at that commit or an existing branch reference can be moved to include it.

Operating Systems (UE24CS242B)

Orange Problem 2 Date: 17/04/2026

Name: Shashidharan VS	SRN: PES2UG24CS568	Section: 4J

6.1: A mark-and-sweep approach starts from branch heads in .pes/refs/heads, traversing all reachable commits, trees, and blobs while marking them in a hash set for fast lookup. After traversal, unmarked objects in .pes/objects are deleted. In large repositories, visited objects can be several times the commit count due to file history.

6.2: Garbage collection is unsafe during commits without coordination. New objects may be created but not yet referenced when GC runs, causing them to be deleted prematurely. This results in broken commits. Systems prevent this using locks, atomic reference updates, and delaying deletion of recent objects to ensure consistency.